

Algorithmen und Datenstrukturen - Programmierbeispiel 3 - Protokoll

1. Algorithmus

Das Programm berechnet den kürzesten Weg zwischen einer Start- und Zielstation im Wiener Verkehrsnetz. Das Programm wird aufgerufen mit: `./Programmname Eingabedatei.txt "Startstation" "Zielstation"` (Programmname.exe für Windows).

Aus der Eingabedatei wird zeilenweise ein Graph aufgebaut. Dabei wird jede Station als Knoten (Node) in einem Array mit Maximalkapazität von 1000 gespeichert und die Verbindungen für jeden Node als Edges mit Fahrzeit und Liniennamen in einer verketteten Liste modelliert. Start- und Zielstation werden aus den String Argumenten in Knotenindizes umgewandelt mit `getNodeIndex()`.

Der Dijkstra-Algorithmus berechnet dann den kürzesten Weg, indem er wiederholt den Knoten mit der kleinsten Distanz aussucht und dann die Distanzen zu dessen Nachbarn aktualisiert, falls sie sich verringert haben. Sobald der Algorithmus beim Ziel ist oder es keinen weiteren vorhandenen Knoten gibt, endet die Berechnung. Es gibt die drei Hilfarrays `dist[]` mit bekannter minimaler Distanz, `visited[]` markiert alle besuchten Knoten und `prev[]` speichert den Vorgängerknoten jedes Knotens für den aktuell besten Weg. Mittels `prev[]` kann dann der gesamte Pfad rekonstruiert werden und zum Schluss Stationsweg, Gesamtfahrzeit und Linienwechsel ausgegeben werden.

2. O-Notation

2.1 createGraph():

Die Datei wird einmal eingelesen und fügt Knoten und Kanten hinzu mittels `findNode()` und `getNodeIndex()`. Für jede Verbindung können mehrere lineare Suchvorgänge notwendig sein. Die Laufzeit beträgt also $O(V * E)$. V ist die Anzahl der Knoten und E die Anzahl der Verbindungen.

2.2 dijkstra():

Um den nächsten Knoten mit minimaler Distanz zu finden, müssen wiederholt alle Knoten linear durchlaufen werden, dadurch ergibt sich eine Laufzeit von $O(V^2 + E) = O(V^2)$.

2.3 reconstructPath():

Beim Rekonstruieren des `prev[]` Arrays müssen maximal alle Knoten besucht werden. Daraus ergibt sich die Laufzeit von $O(V)$.

3. Experimente und Messungen zur Laufzeit

Zur Überprüfung des Programms wurden mehrere Testfälle durchgeführt.

Testfall	Startstation	Zielstation	Fahrzeit	Linienwechsel	Programmzeit in Millisekunden
kurze Strecke	Karlsplatz	Praterstern	5 min	0	14ms
Umstieg	Reumannplatz	Schottenring	10 min	1	16ms
Gegenrichtung	Schottenring	Reumannplatz	10 min	1	12ms
längere Strecke	Hütteldorf	Aspernstraße	35 min	4	15ms
gleiche Station	Karlsplatz	Karlsplatz	0 min	0	12ms
falsche Eingabe	Fakestation	Karlsplatz	Fehlermeldung	-	14ms

Nicht vorhandene Stationen führen zur Fehlermeldung: "Error: Start (oder Target) station '*name*' not found." Um die Zeitmessungen zur Laufzeit durchzuführen, wurde das Programm mit dem Befehl `time ./Programmname Eingabedatei.txt "Startstation" "Zielstation"` geöffnet. Die Laufzeiten lagen bei allen Testfällen im Millisekundenbereich. Unterschiede sind wegen der geringen Eingabegröße kaum messbar.